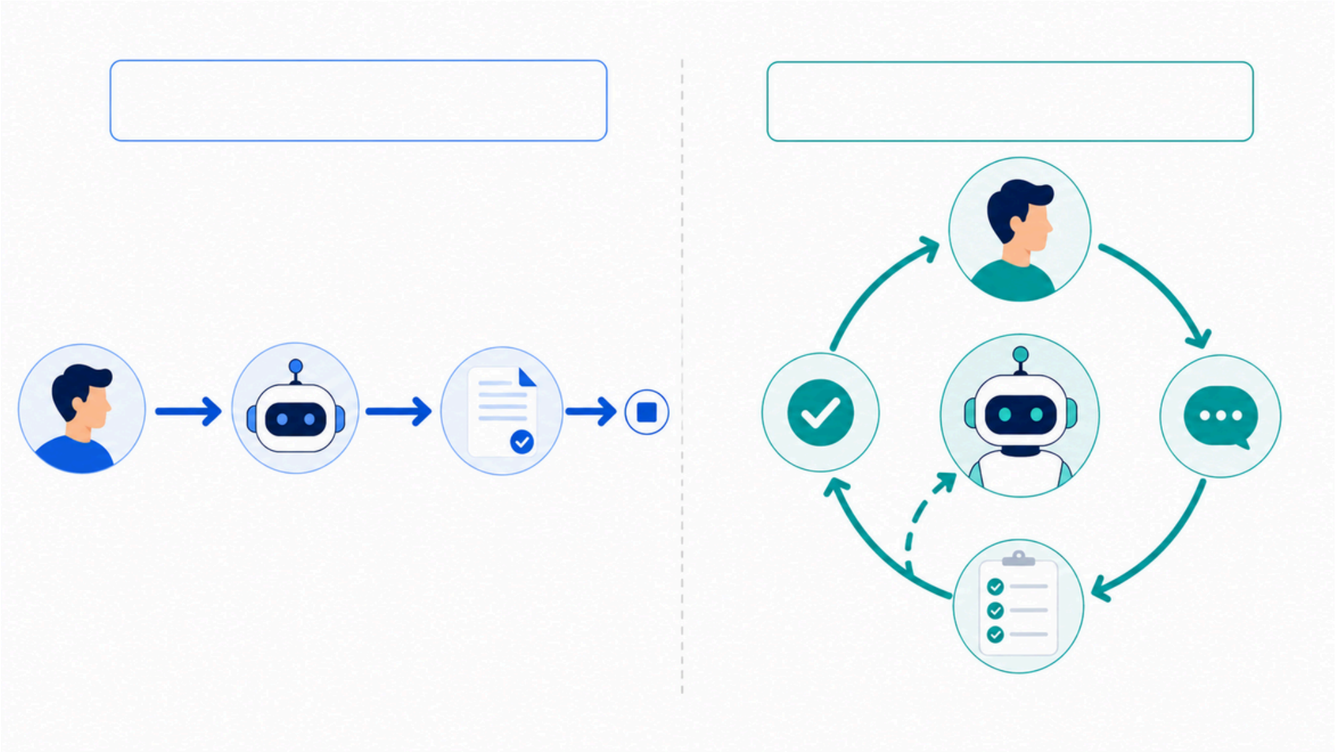
Loop Engineering in Agentic AI

Loop Engineering means designing an AI agent that repeatedly works, checks its result, learns from feedback, and tries again until the goal is achieved.

Instead of a human continuously telling the agent:
“Try again.”
“Fix this problem.”
“Check your answer.”
“Improve the output.”

Why do we need loops?

Because the first answer is not always complete, correct, or usable.



A normal AI call usually gives one response and stops.

That is fine for simple Q&A, but risky when the task has many requirements.

A loop adds a checking stage. If the output fails, feedback goes back into the agent for another attempt.

Simple idea:
Answer once → risky
Check and improve → more reliable

Simple definition

Loop Engineering is the design of the agent’s retry-and-improve cycle.

Loop Engineering means designing an AI agent that can generate work, evaluate the result, receive feedback, try again, and stop when the goal is met.

Generate | Check | Improve | Repeat

Classroom analogy

A student writes. The teacher checks. The student improves. Then the final work is submitted.

Non-loop agent

The agent answers once and stops.

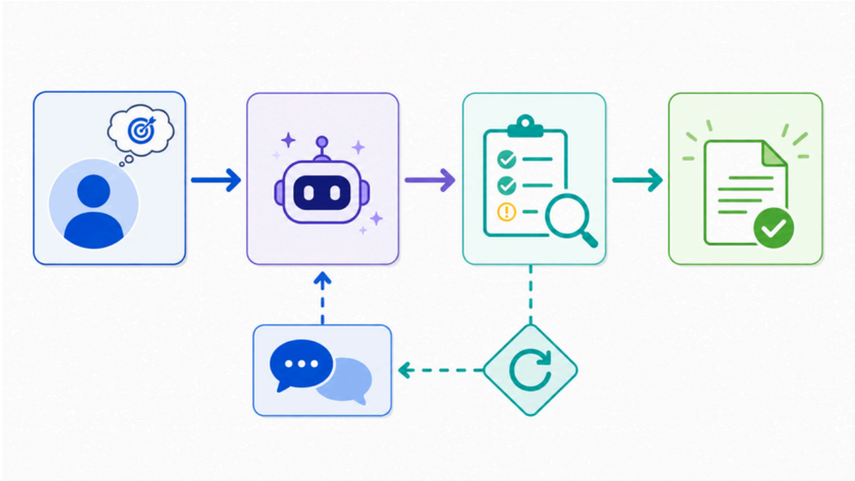
Example: “Write a customer support reply.”

The agent writes one answer. If it forgets the order ID, apology, or replacement offer, the mistake goes directly to the user.

Best for simple tasks:
- Basic question answering
- Drafting ideas
- Low-risk content

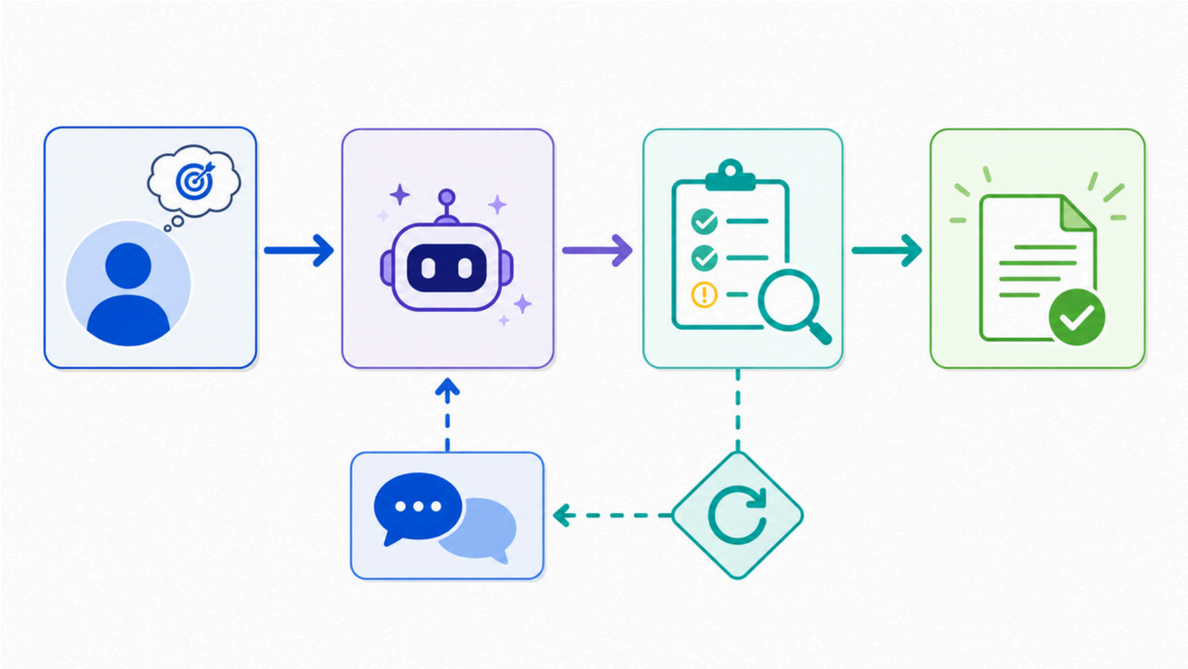
Loop Engineering in Agentic AI

Loop Engineering means designing an AI agent that repeatedly works, checks its result, learns from feedback, and tries again until the goal is achieved.



Loop agent

The agent checks its own work and improves before returning the final answer.

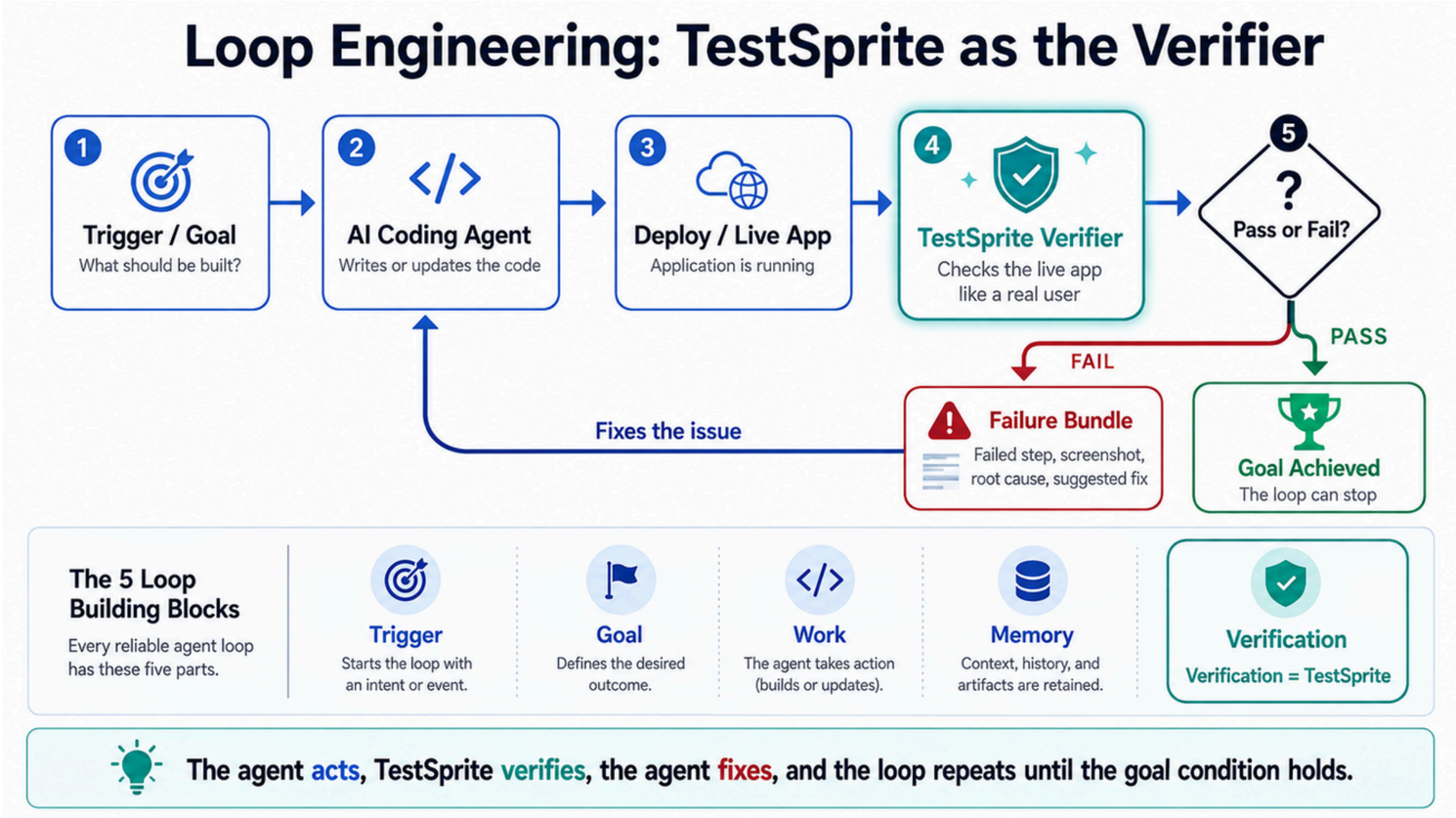


The core cycle

1. Generate an answer
2. Evaluate the answer
3. If it fails, create feedback
4. Try again with feedback
5. Stop when it passes or reaches the limit

Important: A loop must have a stopping rule. Otherwise the agent may keep spending tokens and time without reaching a better result.

Loop agent



The coding agent:

1. Builds or changes the app.
2. TestSprite checks the live app.
3. TestSprite says pass or fail.
4. The agent uses the failure evidence to fix the issue.
5. TestSprite reruns the same verification.

So the loop is:

Agent acts → TestSprite verifies → agent fixes → TestSprite verifies again

Anatomy of a good loop

A reliable loop is built from clear parts, not random retries.

Goal — What final result should the agent achieve?

Action — What will the agent do in each attempt?

Observation — What result, error, or signal comes back?

Evaluation — How do we know the output passed?

Feedback — What should be improved next time?

Stopping Rule — When should the loop finish or fail safely?

Simple formula **Loop = Goal + Action + Evaluation + Feedback + Stop Condition**

Non-loop vs loop agent

Use the right pattern for the right level of risk and complexity.

Non-loop agent

- Answers once
- Fast and simple
- Lower cost
- No automatic self-correction
- Best for low-risk tasks

Speed matters

Loop agent

- Can try multiple times
- Checks output
- Uses feedback to improve
- Higher cost
- Best for multi-step tasks

Quality matters

One-line takeaway

A non-loop agent answers once. A loop agent works, checks, improves, and repeats.

Example 1: Coding agent loop

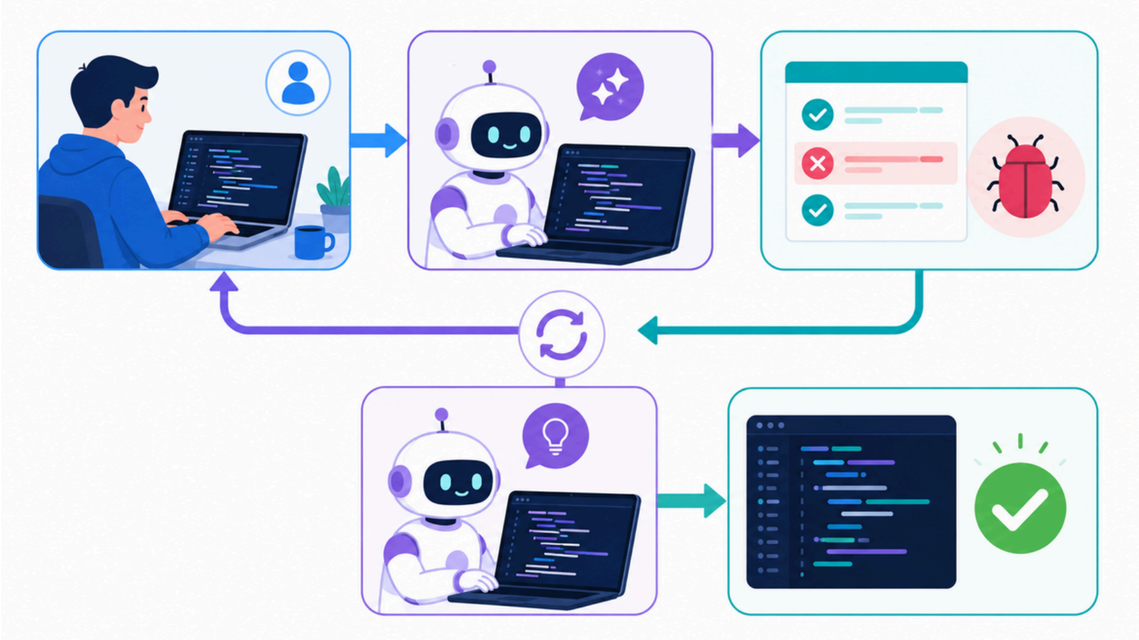
The loop uses test results as feedback.

The agent writes code, runs tests, reads the error, fixes the code, and runs the tests again.

Typical coding loop

Issue → Generate code → Run tests → Read errors → Fix code → Retest → Final patch

Best stopping condition: all tests pass, or the agent reaches the maximum number of repair attempts.



Example 2: Travel planning loop

A real-world loop checks constraints before recommending a plan.

Goal

Create a 3-day trip plan that matches the user's budget, dates, weather preference, hotel needs, and activity style.

Loop checks

- Is it within budget?
- Are dates valid?
- Is travel time realistic?
- Are hotels available?
- Does it follow user constraints?

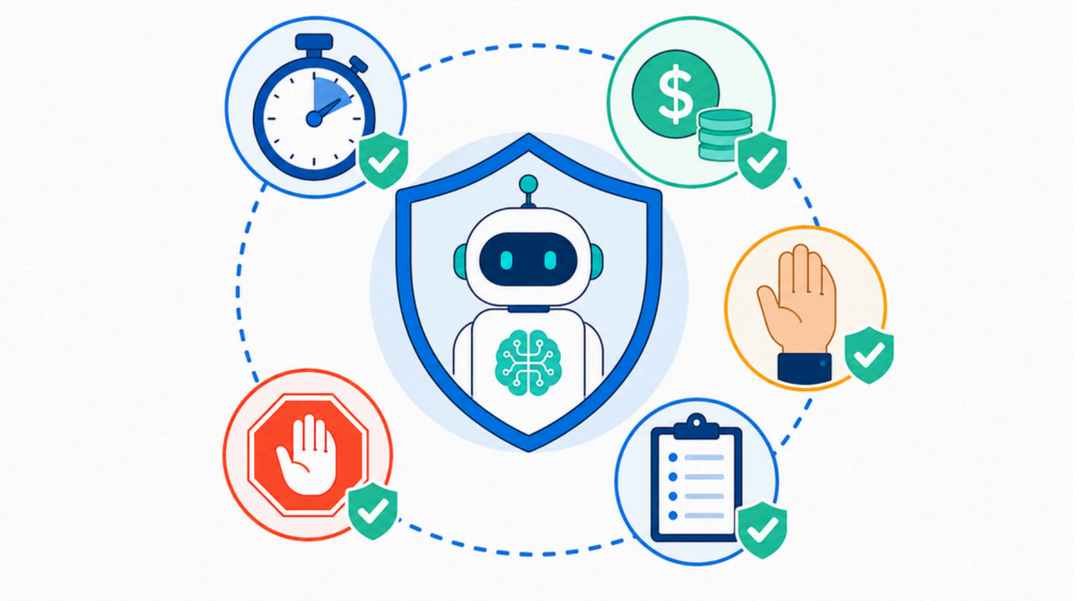
Useful for multi-agent apps: one agent proposes, another checks, and a supervisor decides what to retry.

Guardrails inside the loop

A loop without limits can become expensive, unsafe, or stuck.

Minimum production controls

1. Maximum attempts
2. Token and cost budget
3. Quality evaluator
4. Safety or policy checks
5. Human approval for risky actions
6. Logs for debugging



Key rule

Never design an open-ended agent loop. Every loop needs a stop condition and a fallback path.

Final recap

Loop Engineering is about reliable behavior, not just repeated prompting.

1	**Non-loop**	One answer and stop. Use for simple, low-risk tasks.
2	**Loop**	Work, check, improve, and repeat until the result passes.
3	**Evaluator**	The quality check that decides pass or retry.
4	**Guardrails**	Limits, human approval, logging, and fallback behavior.
5	**Stop condition**	The rule that prevents infinite loops and cost waste.

Main takeaway

Prompt Engineering designs the instruction. Loop Engineering designs what happens after each response.